

Dimensionamiento de Datos

##

¿Qué es y por qué es importante?

###

Definición:

Proceso de estimar el espacio requerido para almacenar datos

Planificación de capacidad de almacenamiento

Optimización de recursos hardware

Objetivos:

Evitar sobrecarga del sistema

Planificar crecimiento futuro

Optimizar costos de infraestructura

Diapositiva 2:

Factores que Influyen en el Dimensionamiento

Factores Clave en el Dimensionamiento

1. Volumen de Datos

Número de registros esperados

Tamaño promedio por registro

Tasa de crecimiento mensual/anual###

2. Operaciones del Sistema

Frecuencia de lecturas/escrituras

Consultas complejas vs simples

Operaciones en tiempo real vs batch

3. Requerimientos de Performance

Tiempos de respuesta esperados

Concurrent users

SLA (Service Level Agreements)

Diapositiva 3:

Fórmulas Básicas de Cálculo

Fórmulas de Dimensionamiento

Cálculo de Espacio para Tablas:

Espacio Total = (Tamaño Registro × N° Registros) + Índices + Overhead

Ejemplo Práctico:

Tabla USUARIOS:

- 100,000 usuarios
- 500 bytes por registro
- Índices: 30% adicional
- Overhead: 10%

Cálculo:

$(500 \times 100,000) = 50 \text{ MB}$

+ Índices (15 MB) + Overhead (5 MB) = **70 MB**

Diapositiva 4:

Tipos de Almacenamiento

Estrategias de Almacenamiento

1. Almacenamiento Relacional

MySQL, PostgreSQL, SQL Server

Estructura fija, ACID compliance

Ideal para datos transaccionales###

2. Almacenamiento NoSQL

MongoDB (documentos)

Redis (clave-valor)

Cassandra (columnar)

Ideal para datos semi-estructurados

3. Almacenamiento en Archivos

Sistema de archivos tradicional

Formatos: JSON, XML, CSV, Parquet

Ideal para logs y datos temporales

Estrategias de Particionamiento

1. Particionamiento Horizontal (Sharding)

Dividir tablas por rangos (ej: por fecha)

Distribuir en múltiples servidores

Mejora performance en lecturas

2. Particionamiento Vertical

Dividir columnas en diferentes tablas

Separar datos frecuentes vs infrecuentes-

Reduce I/O en consultas comunes

3. Particionamiento Híbrido

Combinación de horizontal y vertical

Máxima flexibilidad y optimización

Diapositiva 6:

Técnicas de Indexación

Estrategias de Indexación

Tipos de Índices:

Índices B-tree

: Búsquedas por rango

Índices Hash

: Búsquedas exactas

Índices Full-text

: Búsqueda en texto

Índices Compuestos

: Múltiples columnas

Buenas Prácticas:

Indexar columnas usadas en WHERE, JOIN, ORDER BY

Evitar sobre-indexación (ralentiza INSERTS)

Monitorear uso de índices

SQL vs NoSQL: Elegir la Herramienta Correcta

Bases de Datos Relacionales (SQL)

Ventajas:

ACID compliance

Estructura consistente

Joins complejos

Madurez y estabilidad

Casos de Uso:

Sistemas transaccionales

Datos financieros

Aplicaciones empresariales##

Bases de Datos NoSQL

Ventajas:

Escalabilidad horizontal

Flexibilidad en esquema

Alto rendimiento

Modelos de datos variados

Casos de Uso:

Big Data

Contenido generado por usuarios

Sistemas en tiempo real

Diapositiva 8:

Tipos de Bases de Datos NoSQL

Tipos de Bases de Datos NoSQL

1. Documentales (MongoDB, Couchbase)
2. Almacenan documentos JSON/BSON

Flexibilidad en estructura

Ideal para contenido variable

2. Clave-Valor (Redis, DynamoDB)

Alto rendimiento

Cache y sesiones

Datos simples

3. Columnares (Cassandra, HBase)

Optimizadas para lectura

Big Data analytics

Series temporales

4. Grafos (Neo4j, Amazon Neptune)

Relaciones complejas

Redes sociales

Sistemas de recomendación

Diapositiva 9:

Estrategias de Almacenamiento en Archivos

Sistemas de Archivos Eficientes

1. Estructura de Directorios

/uploads/

├─ users/

| └─ {user_id}/

| | └─ avatars/

| | └─ documents/

| | └─ temp/

├─ products/

| └─ images/

| └─ manuals/

└─ system/

├─ logs/

└─ backups/

2. Nomenclatura de Archivos

- Evitar caracteres especiales
- Usar timestamps para unicidad
- Incluir identificadores de contexto

3. Metadatos y Indexación

- Base de datos para tracking de archivos - Información de tamaño, tipo, propietario - Búsqueda y recuperación eficiente

Diapositiva 10:

Almacenamiento en la Nube

Datos en la Nube: DaaS (Data as a Service)

Servicios de Almacenamiento Cloud:

Amazon S3

: Almacenamiento de objetos

Azure Blob Storage

: Archivos y datos no estructurados

Google Cloud Storage

: Multi-regional, alta disponibilidad

Patrones Comunes:-

Cold Storage

: Datos archivados (más barato)

Hot Storage

: Datos de acceso frecuente

Hybrid

: Combinación según necesidades

Sistemas de Caché para Alto Rendimiento

¿Por qué usar caché?

Reducir latencia de base de datos

Mejorar tiempos de respuesta

Manejar picos de tráfico

Estrategias de Caché:

Cache Aside

: Aplicación maneja cache

Write Through

: Escritura simultánea en cache y DB

Write Back

: Escritura primero en cache

Tecnologías:

Redis

: Estructuras de datos en memoria

Memcached

: Caché simple clave-valor

Varnish

: Caché HTTP

Diapositiva 12:

Estructuras de Datos Especializadas

Estructuras para Casos Específicos

1. Series Temporales

Datos ordenados por tiempo

Optimizadas para consultas por rango

Ejemplo: métricas, logs, datos financieros

2. Datos Geospaciales

Coordenadas y formas geográficas

Consultas de proximidad

Ejemplo: mapas, delivery, real estate

3. Datos de Grafos

Relaciones complejas entre entidades

Consultas de conectividad

Ejemplo: redes sociales, recomendaciones